

HW2 Assessment Report

General Information

Student Information

Field	Value
Student name (printed):	Trương Lý Khải
Student ID:	23127061
Class / Cohort:	23KTPM3
Assignment ID (e.g., HW#00, HW#02):	HW02-AI
Assignment date:	2/6/2026
AI tool(s) used:	Github Copilot
AI tool(s) used:	☒ Yes ☐ No

Self-Assessment Table

No.	Criteria	Grade	Self-Assessed Grade
1	Feature A (Domain + Boundary)	25	25
2	Feature B (Domain + Boundary)	25	25
3	Feature C (Domain + Boundary)	25	25
4	Feature D (Mobile, Domain + Boundary)	15	15
5	Agent Skills	10	5
Total		100	95

Summary of AI Accuracy

Metric	Count	Percentage
Total AI-generated artifacts audited	5	100%
VALID (correct, accepted as-is)	2	40%
INVALID (wrong; rejected)	0	0%
INCOMPLETE (acceptable after edits)	3	60%

Bug Analysis Summary

Severity	Count	Percentage
Severe	0	0.00%
High	5	38.46%
Moderate	8	61.54%
Low	0	0.00%
Total	13	100.00%

Domain Testing and Boundary Value Analysis Application

FR-05: Product listing and search

The general process:

- Step 1: "Trang chủ hiển thị danh sách tất cả sản phẩm dạng lưới (grid). Mỗi sản phẩm hiển thị: **Ảnh** (tỷ lệ chuẩn, có alt text mô tả), **Tên sản phẩm**, **Giá** (đơn vị: đ, định dạng phân cách hàng nghìn)." -> Create 1 test case to check the general UI.
- Step 2: "Thanh tìm kiếm tìm theo tên sản phẩm." -> An input is found, equivalence partitioning can be applied.
- Step 3: "Từ khóa tìm kiếm phải được **hiển thị an toàn** (không render HTML)." -> Partition to 2 classes: search with HTML (invalid) and no-HTML search(valid), the no-HTML class will be further partitioned into 4 valid classes: standard matching search, Unicode matching search, non-matching search and empty search.
- Step 4: Selecting the candidates for the classes: standard matching search (ex: "Keyboard"), Unicode matching search (ex: "Bàn phím"), non-matching search (ex: "zzzzz") and empty search (ex: ""). The invalid class, search with HTML, has a simple script candidate: "<script>alert(1)</script>"
- Step 5: Since a blank search counts as having ≥ 0 blank spaces, we can apply 2-point boundary values here with "" (0 spaces) and " " (1 spaces) respectively, both should still result in the same output.
- Step 6: Identify the output (product list) with the following equivalent classes: an empty list, a full list of all products and a list of only matching products.
- Step 7: For the other requirements, they can be appended to test cases formed by the equivalent classes.

Boundary Value Analysis is generally not applicable here, as only 1 equivalence class has any meaningful order operations ($\leq$ or $\geq$) for the inputs. A total of 8 equivalence classes are made for testing.

FR-11: Order history view (user)

The general process:

- Step 1: Since in general, this requirement does not need any direct input other than adding a new order, I considered using actions done through buttons as "inputs".
- Step 2: Identify 8 actions that can change the state of and order (3 in admin, 3 by user, 1 initial state and 1 action from a foreign user adding to their cart).
- Step 3: Identify 4 possible outputs or history status changes after any input, being: empty, appending a new order, changing the status of an existing order and unchanged.

- Step 4: Create a list of test cases from these classes, for UI elements, they can also be checked during the tests made here.

Boundary Value Analysis is completely not applicable here, as there is no equivalence class with any meaningful order operations ($\leq$ or $\geq$) for the inputs or outputs. For Domain Testing and Equivalence Partitioning, a total of 12 equivalence classes (ECs) are made.

FR-14: Category management (CRUD)

The general process:

- Step 1: There are only 2 main rules for this requirement, one being "Tên danh mục là bắt buộc, không được để trống.", this leads to partitioning our main input, the category name into 2 classes, a valid one: Non-empty name and an invalid one: Empty name.
- Step 2: Similar to FR-05, since a blank search counts as having ≥ 0 blank spaces, we can apply 2-point boundary values here with "" (0 spaces) and " " (1 spaces) respectively, both should still result in the same output.
- Step 3: We can further partition the valid class into 2 different valid class: standard name (e.g. Electronics) and Unicode name (e.g. Máy tính)
- Step 4: Identify the 3 classes for outputs, being: new category, no category or deleted category. The deleted category is added to satisfy the original CRUD requirement of FR-14.

Like in FR-05, Boundary Value Analysis is generally not applicable here, as only 1 equivalence class has any meaningful order operations ($\leq$ or $\geq$) for the inputs. There are also only 6 equivalence classes currently found for this functional requirement.

FR-15: Product management (CRUD)

The general process:

- Step 1: From the requirement, we can identify 5 types of inputs: Name, Price, Description, Image URL and Category. Domain Testing and Equivalence Partitioning can be used on all inputs here.
- Step 2: "Tên sản phẩm: bắt buộc" allows us to partition the current Name class to 2 different ones: blank and not-blank; while "tối đa 255 ký tự" allow us to partition further into: no longer than 255 characters and longer than 255 characters.
- Step 3: For the fact that the amount of characters is specified to be ≤ 255 , we can also apply 2-points boundary values here, with a class when length = 255 and a class when length < 255.
- Step 4: Furthermore, since a blank search counts as having ≥ 0 blank spaces, we can also apply 2-point boundary values here with "" (0 spaces) and " " (1 spaces) respectively, both should still result in the same output.
- Step 5: For the price, it is also noted as "bắt buộc", so we can split them into blank prices and not blank prices. To be precise, we will call blank prices along with other illegal values of prices to be "Not a Number" or NaN prices, while the rest are integer prices.
- Step 6: For integer prices, it is also required that the price "phải là số **dương** (> 0)", meaning that the price being > 0 is a valid class and vice versa.
- Step 7: Since the price being ≤ 0 is an invalid class. We can also apply 2-point boundary values here for value = 0 and value < 0 (e.g. -1)
- Step 8: For image URLs, there are no given rules, however, we can make the participant of the domain into valid link as URLs and invalid unlinked text as URLs.

- Step 9: For description, there are also no given rules, but since description can accept any text or none and still be valid, we partition them into 2 valid classes, empty and non-empty.
- Step 10: The final input is category, and for this we can only choose between No Category (invalid class) and Any Chosen Category (valid class).
- Step 11: Since any text based inputs can be vulnerable to prompt injection, the domain for Name, Image URL, description are partitioned further to accomodate this new class.
- Step 12: After inputs, we can now consider the output class being the product list. Since we also need to test the adjustment and deletion feature for CRUD, 4 output classes can be identified: Product Added, Product List Not Changed, Product Deleted and Product Adjusted.

For this functional requirement in particular, boundary value analysis can be used to great affect here to find important cases that can result in errors within the program. Domain testing is also extremely helpful to make sense of all the inputs and group them into test cases accordingly.

FR-21: Product listing and search for Mobile

The same process in FR-05 applies here. However since Mobile cannot be accessed without directly modifying the code, no actual testing is done for this functional requirement.

AI Conclusion

In general, there exist potential for a well-crafted agent to automate and handle streamlined and complicated tasks. However, creating such an agent is difficult and the average agent most of the time still generates too much errors and defects. The agent skill is useful at the beginning to jumpstart some test case idea with grounded reasoning. Depending on the simplicity of the requirements, the agent may even get the job done first try. The more likely situation, however, is that the user must still manually review each test case manually, while having a general grasp of the relevant knowledge to adjust the batch of tests in the right direction. I can see myself reusing the agent in the future, but due to the usefulness of the agent being limited and the limit of tokens, I do not think it will be used often.

Mandatory Disclosure

"Test cases for FR-05 and FR-14 was initially generated by GitHub Copilot; I reviewed and modified all of those test cases, added edge cases TC05, TC06, TC19. Test cases for FR-11, FR-15, FR-21 and all reports were written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category."

Signature

Field	Value
Student name (printed):	Trương Lý Khải
Student ID:	23127061
Class / Cohort:	23KTPM3
Course:	CS423 / CSC13003 – Software Testing
Instructor:	Mrs. Trần Thị Bích Hạnh

Field	Value
Date:	29/06/2026

Signature:

